Minimizacija Menhetn rastojanja

Staša Đorđević 1007/2025
Matematički fakultet Univerziteta u Beogradu

1 Uvod

Neka je dat skup tačaka $P = \{p_1, p_2, \ldots, p_n\}$ u ravni, gde svaka tačka $p_i = (x_i, y_i)$ ima celobrojne koordinate. Za dve tačke p_i i p_j njihovo *Menhetn rastojanje* definisano je formulom

$$d(p_i, p_j) = |x_i - x_j| + |y_i - y_j|.$$

Menhetn rastojanje predstavlja dužinu najkraće putanje između dve tačke ukoliko je kretanje dozvoljeno samo horizontalno i vertikalno.

U ovom radu razmatramo sledeći problem: za dati skup tačaka potrebno je ukloniti tačno jednu tačku tako da maksimalno Menhetn rastojanje između bilo koje dve preostale tačke bude minimalno.

Formalno, za svaku tačku $p_k \in P$ posmatramo skup

$$P_k = P \setminus \{p_k\}$$

i definišemo

$$D_k = \max_{p_i, p_j \in P_k} d(p_i, p_j).$$

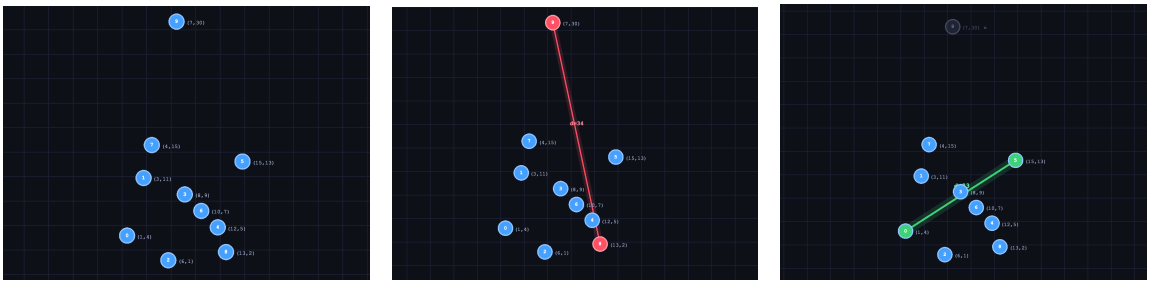
Zadatak je pronaći

$$\min_{1 \le k \le n} D_k.$$

Intuitivno, cilj je ukloniti tačku koja najviše razvlači skup, odnosno tačku čije prisustvo uzrokuje veliko maksimalno rastojanje između nekog para tačaka.

U narednom primeru posmatramo skup od 10 tačaka u ravni. Na slikama vidimo kako tačka koja je značajno udaljena od ostalih određuje maksimalno Menhetn rastojanje u skupu, te kako njenim uklanjanjem to rastojanje znatno smanjujemo.

Primer



(a) Početno stanje skupa (b) Maksimalni par tačaka (c) Nakon uklanjanja tačke

Slika 1: Primer primene algoritma.

2 Naivno rešenje

Naivni pristup podrazumeva da za svaku tačku računamo Menhetn rastojanje do svih ostalih tačaka u skupu, čime možemo odrediti maksimalno rastojanje i indekse tačaka koje ga ostvaruju.

Nakon toga, potrebno je odlučiti koju od te dve tačke ukloniti. To možemo uraditi tako što za svaku od njih posebno izračunamo maksimalno Menhetn rastojanje u skupu koji se dobija izbacivanjem druge tačke. Tačka čijim uklanjanjem dobijamo manju maksimalnu udaljenost predstavlja bolji izbor za uklanjanje.

Međutim, moguće je da već postoji par tačaka sa većim rastojanjem koji ne uključuje nijednu od prethodno posmatranih tačaka. U tom slučaju, izbor između te dve tačke nije od presudnog značaja, jer njihovo uklanjanje ne utiče na globalni maksimum.

Alternativno, sve izračunate udaljenosti možemo sačuvati u pogodnoj strukturi podataka koja preslikava vrednosti rastojanja u indekse tačaka koje ih ostvaruju, pri čemu je struktura uređena po veličini rastojanja. Iz takve strukture možemo najpre uzeti najveću vrednost i pročitati indekse tačaka i i j koje je ostvaruju. Zatim posmatramo sledeću najveću vrednost. Ako ona uključuje neku od tačaka i ili j, tu tačku biramo za uklanjanje. Ukoliko druga najveća vrednost ne uključuje nijednu od njih, izbor između i i j nije bitan.

Ovakav pristup zahteva proveru svih parova tačaka, pa je njegova vremenska složenost $O(n^2)$, što nije optimalno za veće ulazne skupove.

3 Optimalno rešenje

Posmatrajmo dve tačke $p_i = (x_i, y_i)$ i $p_j = (x_j, y_j)$. Njihovo Menhetn rastojanje dato je formulom

$$d(p_i, p_j) = |x_i - x_j| + |y_i - y_j|.$$

Zavisno od međusobnog položaja tačaka u ravni, apsolutne vrednosti se mogu razložiti na četiri slučaja:

$$d(p_i, p_j) = \begin{cases} (x_i - x_j) + (y_i - y_j), & x_i \geq x_j,\ y_i \geq y_j, \\ (x_i - x_j) - (y_i - y_j), & x_i \geq x_j,\ y_i < y_j, \\ -(x_i - x_j) + (y_i - y_j), & x_i < x_j,\ y_i \geq y_j, \\ -(x_i - x_j) - (y_i - y_j), & x_i < x_j,\ y_i < y_j. \end{cases}$$

Grupisanjem članova koji zavise od indeksa i i j dobijamo:

$$d(p_i, p_j) = \begin{cases} (x_i + y_i) - (x_j + y_j), \\ (x_i - y_i) - (x_j - y_j), \\ -(x_i - y_i) + (x_j - y_j), \\ -(x_i + y_i) + (x_j + y_j). \end{cases}$$

Odavde vidimo da se javljaju samo sume i razlike koordinata jedne tačke. Uvedimo sada sledeće oznake:

$$\mathrm{sum}_i = x_i + y_i, \qquad \mathrm{diff}_i = x_i - y_i.$$

Tada se Menhetn rastojanje može zapisati u kompaktnijem obliku:

$$d(p_i, p_j) = \begin{cases} \mathrm{sum}_i - \mathrm{sum}_j, \\ \mathrm{diff}_i - \mathrm{diff}_j, \\ -(\mathrm{diff}_i - \mathrm{diff}_j) = \mathrm{diff}_j - \mathrm{diff}_i, \\ -(\mathrm{sum}_i - \mathrm{sum}_j) = \mathrm{sum}_j - \mathrm{sum}_i. \end{cases}$$

Iz prethodnog izraza sledi da maksimalno Menhetn rastojanje između bilo koje dve tačke u skupu može biti ostvareno ili kao maksimalna razlika vrednosti sum_i, ili kao maksimalna razlika vrednosti diff_i.

Preciznije, važi

$$\max_{i,j} d(p_i, p_j) = \max\left(\max_i \mathrm{sum}_i - \min_i \mathrm{sum}_i,\ \max_i \mathrm{diff}_i - \min_i \mathrm{diff}_i\right).$$

Dakle, problem pronalaženja maksimalnog Menhetn rastojanja svodi se na pronalaženje ekstremnih vrednosti transformisanih koordinata $\mathrm{sum}_i = x_i + y_i$ i $\mathrm{diff}_i = x_i - y_i$, što omogućava rešavanje u linearnom vremenu.

Kad smo našli maksimalno rastojanje (neka ono bude između tačaka sa indeksima i i j), treba videti koju od dve tačke treba ukloniti tako da minimizujemo maksimalno rastojanje skupa tačaka. Jasno je da će uklanjanjem tačno jedne od njih maksimalno rastojanje biti manje ili jednako, nikako veće. Treba proveriti sada da li je manje maksimalno rastojanje koje se dobija kad se ukloni prva tačka ili ono koje se dobije kada se ukloni druga tačka.

4 Implementacija

U ovoj sekciji biće prikazana dva pristupa implementaciji prethodno opisanog algoritma. Prvi pristup koristi uređene strukture podataka i ima vremensku složenost $O(n \log n)$, dok drugi pristup

koristi direktno održavanje ekstremnih vrednosti transformisanih koordinata i omogućava linearnu vremensku složenost $O(n)$.

Oba pristupa zasnivaju se na istom matematičkom uvidu - da se maksimalno Menhetn rastojanje može izraziti preko ekstremnih vrednosti izraza x_i+y_i i x_i-y_i - ali se razlikuju u načinu organizacije podataka i analizi složenosti.

4.1 Implementacija složenosti $O(n \log n)$

Jedan mogući pristup implementaciji je korišćenje uređenih struktura podataka (kao što je `multiset`) za čuvanje transformisanih koordinata tačaka, konkretno:

$$\text{sum}_i = x_i + y_i, \quad \text{diff}_i = x_i - y_i$$

za svaku tačku i.

Ideja je da za svaku tačku privremeno uklonimo njene transformisane vrednosti iz struktura i izračunamo maksimalno Menhetn rastojanje preostalih tačaka. Na taj način možemo proveriti koji izbor tačke za uklanjanje daje minimalnu maksimalnu udaljenost.

Pošto `multiset` održava uređenje, operacije umetanja i uklanjanja zahtevaju vremensku složenost $O(\log n)$, a iteracija kroz sve tačke dodaje faktor n, pa ukupna vremenska složenost ovog pristupa iznosi $O(n \log n)$. Iako ovo rešenje radi, znamo da problem može biti rešen u linearnom vremenu koristeći jednostavnu analizu sumi i razlika bez potrebe za dodatnim uređenim strukturama.

4.2 Implementacija složenosti $O(n)$

Implementacija se zasniva na funkciji `extremePoints`, koja za dati skup tačaka (uz eventualno preskakanje jedne tačke) pronalazi dve tačke koje ostvaruju maksimalno Menhetn rastojanje.

Umesto eksplicitnog računanja svih parova, funkcija prolazi kroz sve tačke i održava sledeće ekstremne vrednosti:

$$\max_i(x_i + y_i), \quad \min_i(x_i + y_i),$$
$$\max_i(x_i - y_i), \quad \min_i(x_i - y_i).$$

Na osnovu ovih vrednosti određuju se kandidati za par tačaka koje ostvaruju maksimalno rastojanje.

Funkcija vraća indekse dve tačke koje daju veću od vrednosti

$$\max_i \text{sum}_i - \min_i \text{sum}_i \quad \text{i} \quad \max_i \text{diff}_i - \min_i \text{diff}_i.$$

U glavnoj funkciji `minimumDistance` postupak je sledeći:

1. Prvo se pronađe par tačaka koje ostvaruju globalno maksimalno rastojanje.
2. Zatim se simulira uklanjanje prve tačke iz tog para i ponovo se računa maksimalno rastojanje.
3. Isto se uradi i za drugu tačku.
4. Kao konačan rezultat uzima se minimum od ta dva dobijena rastojanja.

Funkcija `extremePoints` prolazi kroz sve tačke tačno jednom, pa ima vremensku složenost $O(n)$.
U funkciji `minimumDistance` ova funkcija se poziva tri puta:

- jednom za ceo skup,
- jednom bez prve ekstremne tačke,
- jednom bez druge ekstremne tačke.

Pa je ukupna vremenska složenost $3 * O(n) = O(n)$.
Memorijska složenost je $O(1)$, jer se koristi konstantan broj promenljivih.